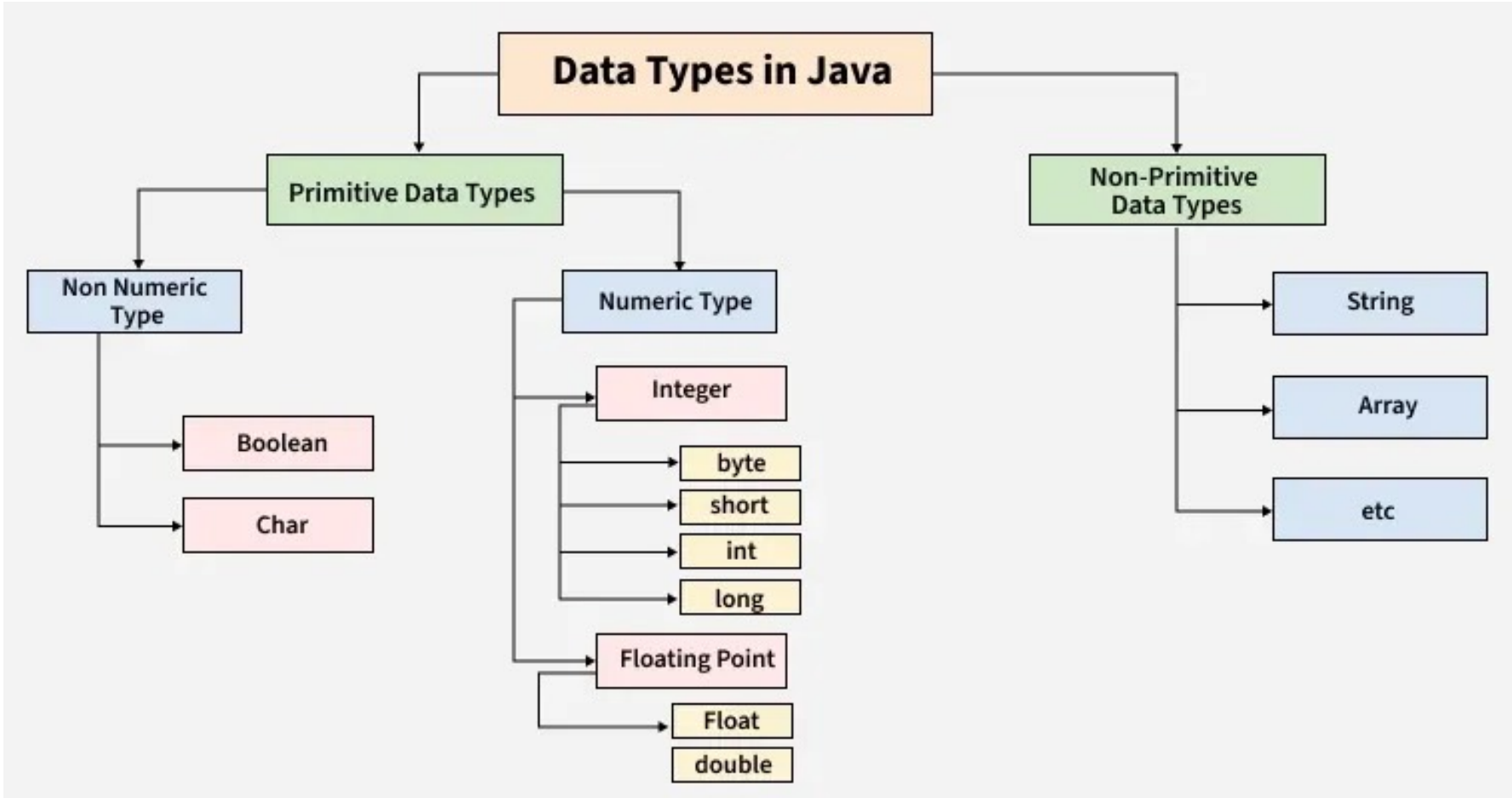
Unit 2

Data Types

OOPJ


Data Types in Java
Primitive Data Types
Non-Primitive Data Types
Non Numeric Type
Numeric Type
String
Integer
Array
Boolean
byte
etc
Char
short
int
long
Floating Point
Float
double

Primitive Data Types

Data – Type	Size	Description
byte	1 byte	Stores whole numbers from -128 to 127
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
short	2 bytes	Stores whole numbers from -32,768 to 32,767
long	8 bytes	Stores whole numbers from -9,223.372,036.854,775.808 to 9,223.372,036,854,775,808
char	2 bytes	Stores a single character/letter
boolean	1 byte	Stores true or false values
float	4 bytes	Stores fractional numbers from 3.4e−038 to 3.4e+038. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers from 1.7e−308 to 1.7e+038. Sufficient for storing 15 decimal digits

BYTE

- Byte stores from -128 and 127.
- Can be used instead of int or other integer types to save memory.

```
byte myNum = 100;
System.out.println(myNum);
```

SHORT

- short stores from -32768 to 32767:

```
short myNum = 5000;
System.out.println(myNum);
```

INT

- int stores from -2147483648 to 2147483647.
- preferred data type when we create variables with a numeric value.

```
int myNum = 100000;
System.out.println(myNum);
```

LONG

- long stores from -9223372036854775808 to 9223372036854775808.
- Used when int is not large enough to store the value.
- you should end the value with an "L":

```
long myNum = 15000000000L;
System.out.println(myNum);
```

FLOAT

- float can store fractional numbers from 3.4e−038 to 3.4e+038.

- should end the value with an "f":

```
float myNum = 5.75f;
System.out.println(myNum);
```

DOUBLE

- Double can store fractional numbers from 1.7e−308 to 1.7e+038.
- you should end the value with a "d":

```
double myNum = 19.99d;
System.out.println(myNum);
```

BOOLEAN

- declared with the boolean keyword
- can only take the values true or false:

```
boolean isJavaFun = true;
boolean isFishTasty = false;
System.out.println(isJavaFun);
System.out.println(isFishTasty);
```

STRING

- The String data type is used to store a sequence of characters (text).
- String values must be surrounded by double quotes:

```
String greeting = "Hello World";
System.out.println(greeting);
```

CHARACTER

- The char data type is used to store a **single** character.
- A char value must be surrounded by single quotes, like 'A' or 'c':

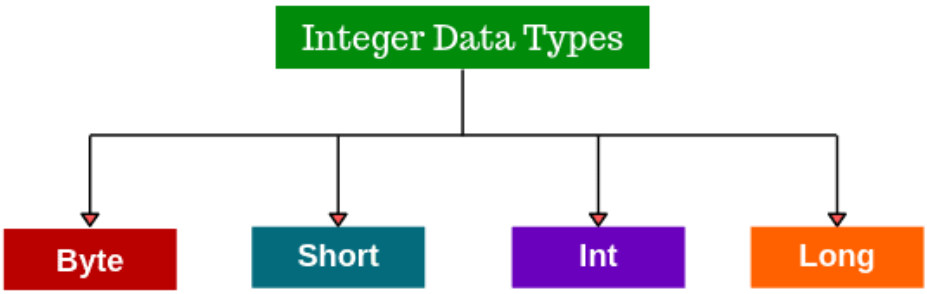
```
char myGrade = 'B';
System.out.println(myGrade);
```

Integer type data:

Integer types of data represent integer number without any fractional parts or decimal points.

For example: age of a person, number of students in a class, distance between planets, distance between galaxy, etc.,

In Java, to store Integer type data we have four data types.



Now the question arises that to store Integer type data why do we require four data types??

Before that let's have a look on a formula

It is important for one to understand that, given a certain amount of memory how much data can be stored inside it. There is a certain maximum value and minimum value that can be stored and the formula to find it is:

*For **n bits**: The minimum value that can be stored is* -2^{n-1}

The maximum value that can be stored is $+2^{n-1}-1$

1) byte is the smallest integer data type which has the least memory size allocated.
Assume, if we create a byte type variable as **byte a;**
It means in the memory **one byte** is allocated and it is referred as **a**.

To convert byte to bits we have, **1 byte = 8 bits.**
And therefore, the minimum value is $-2^{8-1} = -2^7$ = **-128**
The maximum value is $+2^{8-1}-1 = +2^7-1$ = **+127**

Now let's see few examples to understand the above topic.

Example-1:

```
class Demo
{
	public static void main(String[] args)
	{
		byte a;
		a = 127;
		System.out.println(a);
	}
}
```

Output:

127

Example-2:

```
class Demo
{
	public static void main(String[] args)
	{
		byte a;
		a = 128;
		System.out.println(a);
	}
}
```

Output:

Compilation error (because this is out of range).

Example-3:

```
class Demo
{
	public static void main(String[] args)
	{
		byte a;
		a = -128;
		System.out.println(a);
	}
}
```

Output:

-128

Example-4:

```
class Demo
{
        public static void main(String[] args)
        {
                byte a;
                a = -129;
                System.out.println(a);
        }
}
```

Output:

Compilation error (because this is out of range).

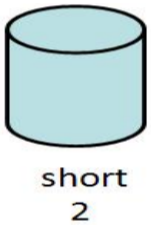
Which means, one value more than +127 or one value less than -128 cannot be stored but any value between the range of -128 to +127 can be stored using a byte type variable.

Therefore, the age of a person, the current month of the year etc., can be stored using byte data type.

2) short is a signed 16-bit byte.

Assume, if we create a short type variable as **short a;**

It means in the memory **two bytes** is allocated and it is referred as **a**.



Converting byte to bits: **2 bytes = 16 bits.**

And therefore, the minimum value is $-2^{16-1} = -2^{15}$ = **-32768**

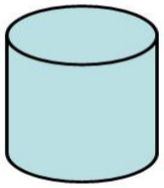
The maximum value is $+2^{16-1} - 1 = +2^{15}$-1 = **+32767**

Which means, one value more than +32767 or one value less than -32768 cannot be stored but any value between the range of -32768 to +32767 can be stored using a short type variable.

Therefore, salary of a fresher, height of Mount Everest etc., can be stored using short data type.

3) int is a signed 32-bit byte. It is the most commonly used integer type. In addition to other uses, variables of type int are commonly employed to control loops and to index arrays.

Assume, if we create an int type variable as **int a;**
It means in the memory **four bytes** is allocated and it is referred as **a**.

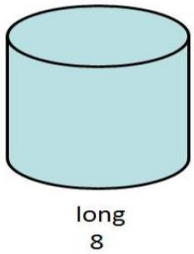


Converting byte to bits, **4 bytes = 32 bits.**
And therefore, the minimum value is $-2^{32-1} = -2^{31}$ = **-2,147,483,648**
The maximum value is $+2^{32-1} - 1 = +2^{31}$-1 = **+2,147,483,647**

Which means, one value more than +2,147,483,647 or one value less than -2,147,483,648 cannot be stored but any value between the range of -2,147,483,648 to +2,147,483,647 can be stored using an int type variable.

Therefore, distance between planets etc., can be stored using int data type.

4) long is a signed 64-bit byte and is useful for those occasions where an int type is not large enough to hold the desired value. The range of long is quite large. This is useful, when big whole numbers are needed.
Assume, if we create an long type variable as **long a;**
It means in the memory **eight bytes** is allocated and it is referred as **a**.



Converting byte to bits, **8 bytes = 64 bits.**
And therefore, the minimum value is $-2^{64-1} = -2^{63}$ = **-9,223,372,036,854,775,808**
The maximum value is $+2^{64-1} - 1 = +2^{63}$-1 = **+9,223,372,036,854,775,807**

Which means, any value between the range of -9,223,372,036,854,775,808 to +9,223,372,036,854,775,807 can be stored using a long type variable.

Therefore, distance between galaxy etc., can be stored using long data type.

Now let's look an example on long data type.

```
class Demo
{
        public static void main(String[] args)
        {
                long a;
                a = 9,223,372,036,854,775,807;
                System.out.println(a);
        }
}
```

In Java, there is rule that whenever we want a value to be stored as long, at the end of the value a suffix of 'L' must be given.

Output:

Compilation error.

Hope now we have got the answer for our question of having four data types.

Integer values exists in different quantities, to handle different quantities of integer data different data types has been provided.

How Integer data is stored in the memory?

→The below is an example of how positive number is stored.

byte a = 24;

2	24	
2	12	- 0
2	6	- 0
2	3	- 0
	1	- 1

This type of conversion is called base-2 format.

0	0	0	1	1	0	0	0

MSB = 0 → positive
1 → negative

→The below is an example of how negative number is stored.

byte a = -24;

For this, we require base-2 format + 2's compliment of a number

We know the base-2 value of 24 is **0001100.**

Now, let's find the 2's compliment of the number. **0 0 0 1 1 0 0 0**

Step-1: Find 1's compliment of the number by converting 0's to 1's and vice versa. **1 1 1 0 0 1 1 1**

Step-2: Find 2's compliment by adding 1 to the number. **+ 1**

This is the 2's compliment of the number→ **1 1 1 0 1 0 0 0**

1	1	1	0	1	0	0	0

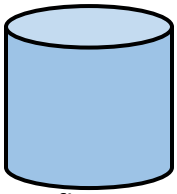
MSB = **1 → negative**

Real number type data:

In the real world we are rarely surrounded by Integer type data but often come across decimal numbers. For example, weight: 56.4kg, Temperature: 28.5 degrees, distance: Rajajinagar HO to metro station 0.5km, etc.

In Java to store such real number data types we have float, double.

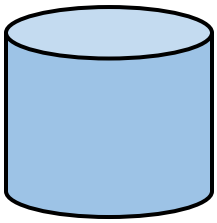
1) **float** The float data type is a single-precision **32-bit** IEEE 754 floating-point. Float data type in Java **stores a decimal value** with **6-7 total digits of precision**. So for example 12.12345 can be saved as a float, but 12.123456789 cannot be saved as the float. When representing a float data type in Java we should **append the letter f** to the end of the data type, otherwise, it will save as double.
   Assume, if we create an float type variable as **float a;**
   It means in the memory **four bytes** is allocated and it is referred as **a**.



float
4

Range: $-34e^{38}$ to $34e^{38}$

2) **double** The double data type is a double-precision **64-bit** IEEE 754 floating-point. The double data type is normally the **default choice for decimal values**. The data type should never be used for precise values, such as currency. Double data type stores decimal values with **15-16 digits of precision**. The default value is 0.0d, this means that if you do not append f or d to the end of the decimal, the value will be stored as a double in Java.
Assume, if we create an double type variable as **double a;**
It means in the memory **eight bytes** is allocated and it is referred as **a**.



double
8

Range: $-1.7e^{308}$ to $1.7e^{308}$

Now let's see few examples to understand the above topic.

Example-1:

```
class Demo
{
        public static void main(String[] args)
        {
                double a=45.5;
                System.out.println(a);
        }
}
```

Output:

45.5

Example-2:

```
class Demo
{
        public static void main(String[] args)
        {
                float a=45.5;
                System.out.println(a);
        }
}
```

In Java, there is rule that whenever we want a value to be stored as float, at the end of the value a suffix 'f' must be given, otherwise it is considered to be a double number.

Output:

Compilation error.

Solution:

Method 1

```
class Demo
{
        public static void main(String[] args)
        {
                double a=(float)45.5;
                System.out.println(a);
        }
}
```

Method 2

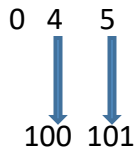
```
class Demo
{
        public static void main(String[] args)
        {
                double a=45.5f;
                System.out.println(a);
        }
}
```

Literals: In java values are called as literals.

Now let's see few examples to understand literals.

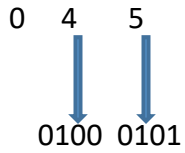
1) int a=45; ←Decimal literal
   S.O.P (a); // Output is 45.

2) int a=045; ←Octal literal
   S.O.P (a); // Output is 37.
   Note: Here 0 acts as the prefix to the literal and converts it from decimal to octal literal.



Now we have 100101 as our number, wherever 1 is present we take 2^{index} and add all.
We get **$2^0+2^2+2^5 = 1+4+32 = 37$**

3) int a=0x45;
   S.O.P (a); //Output is 69.
   Note: Here 0x acts as the prefix to the literal and converts it from decimal to hexadecimal literal.



Now we have 01000101 as our number, wherever 1 is present we take 2^{index} and add all.
We get **$2^0+2^2+2^6 = 1+4+64 = 69$**

4) int a= 0b100101;
   S.O.P (a); // Output is 37.
   Note: Here 0b is a prefix for binary literals.

Wherever 1 is present we take 2^{index} and add all. As seen previously we'll get 37.

Let us move ahead and learn about the next data type.

Character type data:  $ *1* ***X***

Had a look on the above characters? 

Well they all fall under character type data but, computer doesn't understand ***A@#*14***. It only understands 0's and 1's and hence conversion should happen. Before understanding that let us first know the different characters.
Let us consider four symbols ***A B C D*** but, none of these symbols can your computer understand because all it understands is binary numbers 0's and 1's. So, let us attach a binary code to each of these symbols like this 

SYMBOLS	CODE
A	**00**
B	**01**
C	**10**
D	**11**

Let us consider if there were 8 symbols and look at its binary code.

SYMBOLS	CODE
A	**000**
B	**001**
C	**010**
D	**011**
E	**100**
F	**101**
G	**110**
H	**111**

As you can see, as the number of symbols increase their code size also increases.

Can you guess the code size for 16 characters? 

SYMBOLS	CODE	SYMBOLS	CODE
A	0000	I	1000
B	0001	J	1001
C	0010	K	1010
D	0011	L	1011
E	0100	M	1100
F	0101	N	1101
G	0110	O	1110
H	0111	P	1111

If you are focusing then you might have noticed there is a mathematical relation between **no. of symbols** and **code length.**

Let us consider four symbols  **A B C D**

SYMBOLS	CODE
A	00
B	01
C	10
D	11

NO. Of Symbols

4

2^2

Similarly let us consider eight symbols  **A B C D E F G H**

SYMBOLS	CODE
A	000
B	001
C	010
D	011
E	100
F	101
G	110
H	111

NO. Of Symbols

8

2^3

If you are focusing on the **power of 2,** you can see it is only the **code length.**

Let us consider one more case of 16 symbols to understand this.

16 Symbols  **A B C D E F G H I J K L M N O P**

SYMBOLS	CODE	SYMBOLS	CODE
A	0000	I	1000
B	0001	J	1001
C	0010	K	1010
D	0011	L	1011
E	0100	M	1100
F	0101	N	1101
G	0110	O	1110
H	0111	P	1111

NO. Of Symbols

16

2^4

Code Length - 4

Now you understood the relation between code length and number of symbols. Like this Americans found **128 symbols** and gave the name as **ASCII**. But Java does not follow ASCII as it only consist of English symbols. Have a look at the ASCII table below.

Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value
00	NUL	10	DLE	20	SP	30	0	40	@	50	P	60	`	70	p
01	SOH	11	DC1	21	!	31	1	41	A	51	Q	61	a	71	q
02	STX	12	DC2	22	"	32	2	42	B	52	R	62	b	72	r
03	ETX	13	DC3	23	#	33	3	43	C	53	S	63	c	73	s
04	EOT	14	DC4	24	$	34	4	44	D	54	T	64	d	74	t
05	ENQ	15	NAK	25	%	35	5	45	E	55	U	65	e	75	u
06	ACK	16	SYN	26	&	36	6	46	F	56	V	66	f	76	v
07	BEL	17	ETB	27	'	37	7	47	G	57	W	67	g	77	w
08	BS	18	CAN	28	(	38	8	48	H	58	X	68	h	78	x
09	HT	19	EM	29	)	39	9	49	I	59	Y	69	i	79	y
0A	LF	1A	SUB	2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
0B	VT	1B	ESC	2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
0C	FF	1C	FS	2C	,	3C	<	4C	L	5C	\	6C	l	7C	|
0D	CR	1D	GS	2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
0E	SO	1E	RS	2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
0F	SI	1F	US	2F	/	3F	?	4F	O	5F	_	6F	o	7F	DEL

ASCII stands for **American standard code for information interchange.**
It is a **7-bit** binary representation for 128 symbols.
Java does not follow ASCII as it is a English biased language and does not support symbols of other languages.
Hence java follows **UNICODE** which provides binary representation for **65,536** symbols of commonly spoken languages across the world.
It is a **16-bit** code and hence a char variable in java takes **2 bytes** of memory.

Let us write a simple code to print character type data

```
Class Demo
{
        Public static void main(String[] args)
        {
                Char ch = 'a';
                System.out.println(ch);
        }
}
```

Output: a

Let us now look at the last data type that is **boolean** data type.

Boolean data type:

To store yes/no type data or true/false type data, java provides boolean data type. Size of this data type is decided by JVM and we have already learned JVM is platform dependent, hence the size of this data type will differ depending on the type of operating system.

The remaining types of data that is audio, video and still pictures are handled using built in libraries.

Type casting in java:

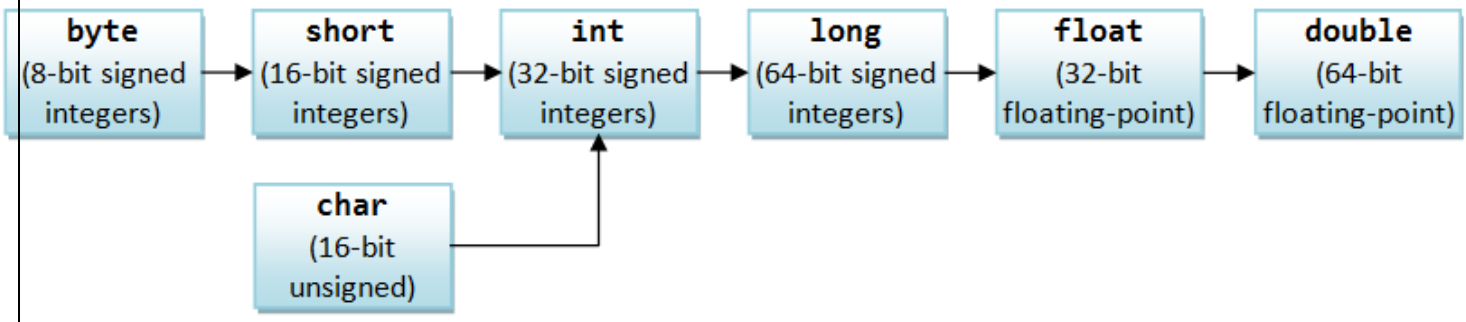
Now you wonder what is type casting?

Well let me tell you, **type casting is a process of converting one type of data to another.**

In Java, there are two types of casting:
Implicit casting (automatically) - converting a smaller type to a larger type size
byte -> short -> char -> int -> long -> float -> double.
Explicit casting (manually) - converting a larger type to a smaller size type.

Implicit type casting:

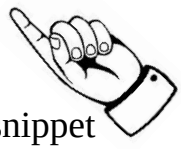
When a smaller data type is converted to a larger data type, the conversion is automatically performed by **the java complier** and is referred to as implicit type casting.
Advantage: No loss of precision.
Consider the **Implicit type casting chart** given below to understand this:



Orders of Implicit Type-Casting for Primitives

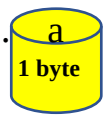
Let us consider a code snippet to understand this:

```
byte a = 45;
double b;
 b = a;
```

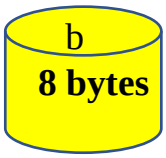


let us understand implicit type casting using the above code snippet

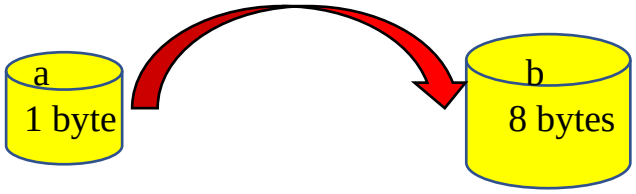
a is a variable of type byte whose size is 1 byte.



b is a variable of type double whose size is 8 bytes.



b = a; we are now trying to store the data present in **a into b.**
a is of type byte and can store 1 byte. b is of type double and can store 8 bytes.
we are trying to store data of smaller size into larger size.



This conversion is implicitly done without user interaction and hence it is referred to as implicit type casting.

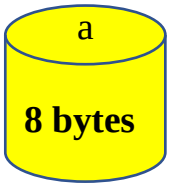
Explicit type casting:

When a larger data type is converted to a smaller data type, the conversion **not automatically performed** by **the java complier** and must be done by **programmer explicitly** and hence it is referred to as implicit type casting.

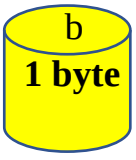
Let us consider a simple code snippet to understand this, the way we understood implicit type casting.

```
double a = 45.5;
byte b;
 b = a;  → error
```

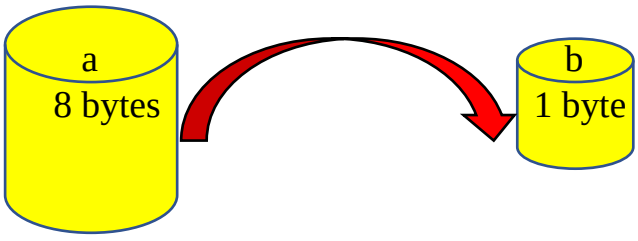
a is a variable of type double whose size is 8 bytes.



b is a variable of type byte whose size is 1 byte.



b = a; will give you ***error*** as you are trying to store a larger type of data into smaller type.



The above conversion will result in error as **loss of precision** occurs.
To get the error free output, we have to explicitly convert the data as shown below

```
double a = 45.5;
byte b;
b = byte(a);
```

b is of type byte and it will only store 45 and 0.5 is lost during the conversion which is the disadvantage of explicit type casting.

STRINGS

Unlike other programming languages, Strings in java are different. Well how you wonder? Let us first understand what is a string.

C → ***is this a single character or multiple character?***

If you can recollect, it is a single character. Now let me ask you the same question by considering one more example as shown below:

CODE → ***is this a single character or multiple character?***

Now you will answer, **it is a sequence of characters or collection of character or series of characters.**

The different words you have used to answer the second example is only referred to as **STRINGS** in java.

You are able to differentiate between a **character** and **a string**. But computer can't tell the difference between character and string.
To resolve this **problem**, java came up with a **solution.**
Well what you under?

The solution is:

`Put the characters in single quotes` → 'C'

`Put the string in double quotes` → "CODE"

When the computer encounters single quotes, it treats it as character. Similarly, when the computer encounters double quotes, it treats it as stings .

What is a Character?

A Character is a value enclosed within single quotes.

What is a String?

A String is a series of characters enclosed within double quotes.
Strings are treated as objects in java.

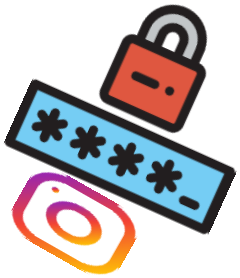
In java, Strings are further divided into two types:

1) **Mutable Strings**
2) **Immutable Strings.**

As the name itself indicates, **mutable strings** are such strings which are **changeable** and **immutable strings** are such strings which are **non-changeable**.

Let us look at few real time examples to understand this:

Months in a year



Instagram Password

MUTABLE STRINGS

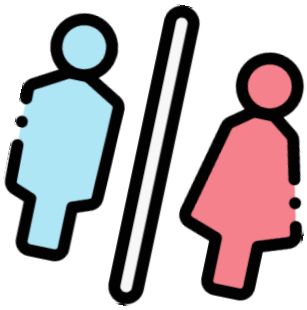
Gmail id

Passport Details.

If you look at the examples given above, you may notice they are all changeable and hence they fall under the category of Mutable Strings.

Let us now have a look at non-changeable strings:

Earths colour



Gender

IMMUTABLE STRINGS

Date-of-birth

Flags colour Combination

Similarly, if you look at the examples given above, you may notice they are all non-changeable and hence they fall under the category of Immutable Strings.

Let us now learn Immutable strings in detail.

Definition: *Immutable strings are such strings that are unmodifiable or unchangeable.* ***They are Created using String Class.***

Different ways of creating Immutable strings:

1) **char name[ ] = { 'j' , 'a' , 'v' , 'a' }**
   String s = new String(name);
2) **String s = "java";**
3) **String s = new String ("java");**

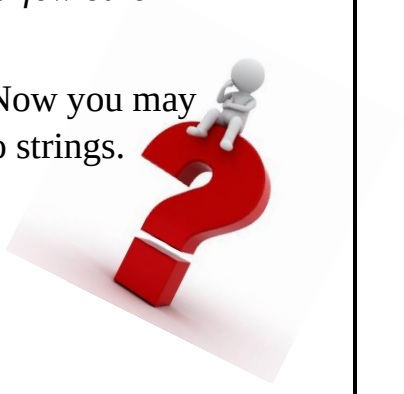
After getting to know different ways of creating strings, one most now learn different ways of comparing strings.

Different ways of comparing strings:

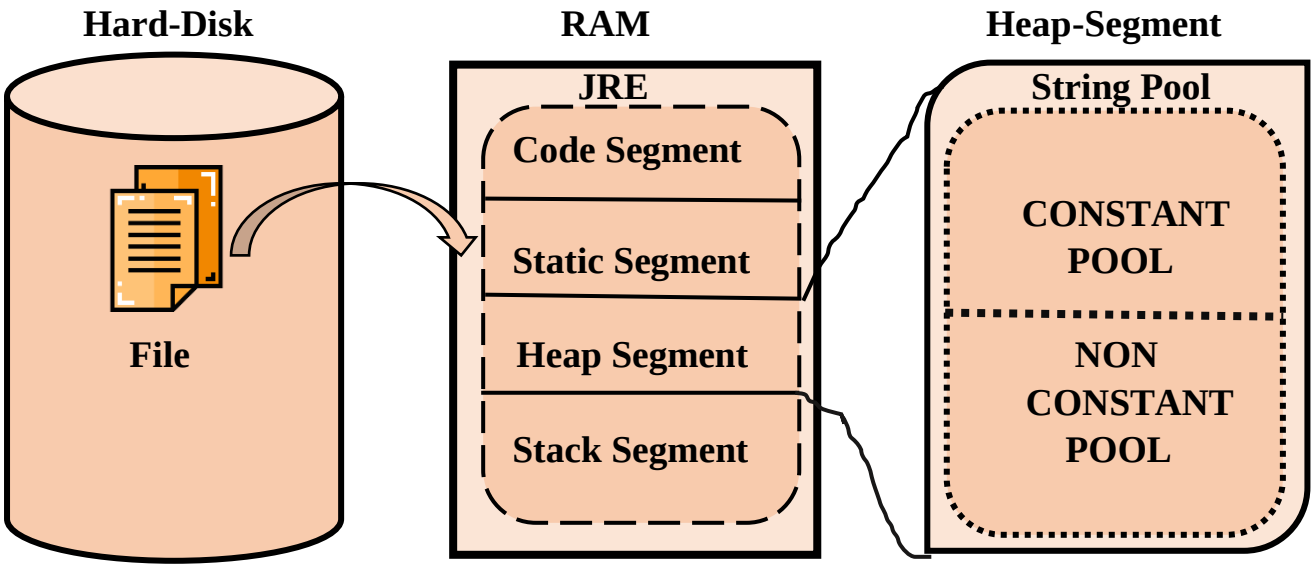
1) **==** → **String references are compared.**
2) **equals()** → **String values/literals are compared.**
3) **CompareTo()** → **Strings are compared character by character.**
4) **equalsIgnoreCase() ()** → **Strings are compared by ignoring the case.**

*Before we start writing our first code on strings, let us understand few other concepts which one should be aware of while coding strings***.**

We have already discussed hard-disk, ram, and microprocessor. Now you may wonder why am i introducing them here and how is this related to strings.



Consider the diagram shown below:



We know microprocessor is the most important device on our computer and there are two other devices which play a crucial role when it comes to storing the data which are hard-disk and ram. The file is stored on hard-disk but if it has to execute it should be placed on ram. Also, we have learned, when the file is placed on ram, the entire region is not allocated for its execution instead, one region of ram is allocated for this program file to execute which is only referred to as **JRE.**

JRE is further divided into four segments:
1) Code Segment
2) Static Segment
3) Heap Segment
4) Stack Segment.

Out of all those segments, **Strings** are allocated memory on **Heap Segment**.

Heap Segment further consists of two pools:
1)Constant pool
2)Non-Constant Pool.

To understand strings from memory perspective, one must know the features of these pools which is shown below:

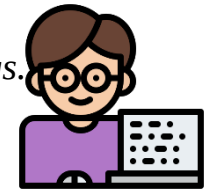
CONSTANT POOL

Strings that are created **without using new** keyword are allocated memory on this pool.
Duplicates are not allowed.

Non-CONSTANT POOL

Strings that are created **using new** keyword are allocated memory on this pool.
Duplicates are allowed.

The wait is over, you now have sufficient knowledge to start coding strings.



Example 1)

```
class Demo
{
    public static void main(String[] args)
    {
        String s1 = "JAVA";
        String s2 = "JAVA";
        if(s1 == s2)
        {
            System.out.println("References are equal");
        }
        else
        {
            System.out.println("References are not equal");
        }
    }
}
```

Output: References are equal.

Explanation: In the above code we made use of "==" operator to compare two strings. We know "==" operator compares two strings based on the references. Both the strings are created without using new keyword and if you recollect, they will now be allocated memory on constant pool where duplicates are not allowed. Hence only one copy of string literal is created and both the references will point to the same string which means both s1 and s2 will have same address as only one string object is created. Thus, the output references are equal.

Example 2)

```
class Demo
{
    public static void main(String[] args)
    {
        String s1 = "JAVA";
        String s2 = "JAVA";
        if(s1.equals(s2) == true)
        {
            System.out.println("String values are equal");
        }
        else
        {
            System.out.println("String values are not equal");
        }
    }
}
```

Output: String values are equal.

Explanation: In the above code we made use of "equals ()" to compare two strings. We know equals() compares two strings based on the values. Both the strings are created without using new keyword and if you recollect, they will now be allocated memory on constant pool where duplicates are not allowed. Hence only one copy of string literal is created. Since both the strings are same, we get the output as String values are equal.

Example 3)

```
class Demo
{
    public static void main(String[] args)
    {
        String s1 = "JAVA";
        String s2 = new String("JAVA");
        if(s1 == s2)
        {
            System.out.println("References are equal");
        }
        else
        {
            System.out.println("References are not equal");
        }
    }
}
```

Output: References are not equal.

Explanation: In the above code we made use of "==" operator to compare two strings. We know "==" operator compares two strings based on the references. String s1 is created without using new keyword so it gets allocated memory on constant pool and String s2 is created using new keyword so it gets allocated memory on non-constant pool. So now, two different string objects are created with two different addresses. S1 and s2 are the references pointing to two string objects hence they will now consist of different addresses and thus we get the output as references are not equal.

String Comparison in Java

We can compare string in java on the basis of **content** and **reference**.

It is used in **authentication** (by equals() method), **sorting** (by compareTo() method), **reference matching** (by == operator) etc.

There are three ways to compare string in java:

- By = = operator
- By equals() method
- By compareTo() method

Let's start with understanding == operator:

The = = operator compares **references not values**.

Example1

```
class Demo
{
    public static void main(String[] args)
    {
        String s1 = new String("JAVA");
        String s2 = new String("JAVA");

        if(s1==s2)
        {
            System.out.println("String references are equal.");
        }
        else
        {
            System.out.println("String references are not equal.");
        }
    }
}
```

Output: **String references are not equal.**

Whenever **new** keyword is used to create string, it gets created in non-constant pool. And when == operator is used to compare two strings its references are compared and not the values. And as s1 has different reference than s2, we get the output as references are not equal.

Now let's see comparison by equals() method:

The String equals() method compares the original content of the string. It compares values of string for equality. String class provides two methods:

- **public boolean equals(Object another)** compares this string to the specified object.
- **public boolean equalsIgnoreCase(String another)** compares this String to another string, ignoring case.

Example2

```
class Demo
{
    public static void main(String[] args)
    {
        String s1 = new String("JAVA");
        String s2 = new String("JAVA");

        if(s1.equals(s2)==true)
        {
            System.out.println("String references are equal.");
        }
        else
        {
            System.out.println("String references are not equal.");
        }
    }
}
```

Output: **String values are equal.**

Example3

```
class Demo
{
    public static void main(String[] args)
    {
        String s1 = "java";
        String s2 = new String("JAVA");

        if(s1.equals(s2)==true)
        {
            System.out.println("String values are equal.");
        }
        else
        {
            System.out.println("String values are not equal.");
        }
    }
}
```

Output: **String values are not equal.**

Example4


```
class Demo
{
    public static void main(String[] args)
    {
        String s1 = "java";
        String s2 = new String("JAVA");

        if(s1.equalsIgnoreCase(s2)==true)
        {
            System.out.println("String values are equal.");
        }
        else
        {
            System.out.println("String values are not equal.");
        }
    }
}
```

Output: **String values are equal.**

Example5

```
class Demo
{
    public static void main(String[] args)
    {
        String s1 = "JAVA";
        String s2;
        s2 = s1;
        if(s1==s2)
        {
            System.out.println("references are equal.");
        }
        else
        {
            System.out.println("references are not equal.");
        }
    }
}
```

Output: **references are equal.**

Here as new keyword is not used s1 gets created in constant pool. And s2 is reference which gets created in constant pool. So s1 reference is assigned to s2, which means both are referring to same value.

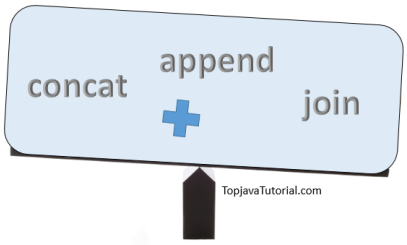
String Concatenation in Java

In java, string concatenation forms a new string *that is* the combination of multiple strings. There are two ways to concat string in java:

- By + (string concatenation) operator
- By concat() method

String Concatenation by + (string concatenation) operator

Java string concatenation operator (+) is used to add strings.



Example6

```
class Demo
{
    public static void main(String[] args)
    {
        String s1 = "JAVA";
        String s2 = "PYTHON";
        String s3 = "JAVA"+"PYTHON";
        String s4 = "JAVA"+"PYTHON";

        if(s3==s4)
        {
            System.out.println("references are equal.");
        }
        else
        {
            System.out.println("references are not equal.");
        }
    }
}
```

Output: **references are equal.**

As here values are getting added and not the references. And duplication cannot happen in constant pool, therefore references are same.

Example7

```
class Demo
{
    public static void main(String[] args)
    {
        String s1 = "JAVA";
        String s2 = "PYTHON";
        String s3 = s1+s2;
        String s4 = s1+s2;

        if(s3==s4)
        {
            System.out.println("references are equal.");
        }
        else
        {
            System.out.println("references are not equal.");
        }
    }
}
```

Output: **references are not equal.**

As here values are not getting added but references are, because s1 and s2 are not literals. And duplication is possible in non-constant pool, therefore references are not the same.

String Concatenation by concat()

The String concat() method concatenates the specified string to the end of current string.

Syntax: **public** String concat(String another)

Let's see the example of String concat() method.

Example9

```
class Demo
{
    public static void main(String[] args)
    {
        String s1 = "java";
        String s2 = "python";
        String s3 = s1.concat(s2);
        System.out.println(s3);

    }
}
```

Output: **javapython**

Although it is important to note that Strings are immutable. Now lets see what does this mean.

Immutable String in java

In java, **string objects are immutable**. Immutable simply means unmodifiable or unchangeable.

Once string object is created its data or state can't be changed but a new string object is created.

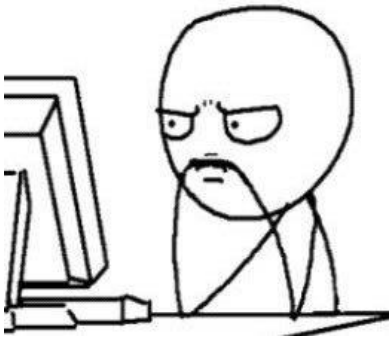
Let's try to understand the immutability concept by the example given below:

Example10

```
class Demo
{
    public static void main(String[] args)
    {
        String s1 = "java";
        String s2 = "python";
        System.out.println(s1);
        System.out.println(s2);
        s1.concat(s2);
        System.out.println(s1);
        System.out.println(s2);
    }
}
```

Output:

```
java
python
java
python
```



In java whenever an immutable string is attempted to be modified then that string will not be modified, instead another string is created in non-constant pool with new reference. And as of now no object is assigned to store the new reference, which is why we cannot see the concatenated string.

So now let's see how to get the concatenated string…

```
class Demo
{
    public static void main(String[] args)
    {
        String s1 = "java";
        String s2 = "python";
        System.out.println(s1);
        System.out.println(s2);
        s1 = s1.concat(s2);
        System.out.println(s1);
        System.out.println(s2);
    }
}
```

Output:

```
java
python
javapython
python
```

Previously we had a reference for the concatenated string but we did not assign it to anything, whereas here we are assigning it back to s1 so that now s1 is pointing to the new concatenated string.

Why string objects are immutable in java?

Because java uses the concept of string literal. Suppose there are 5 reference variables, all refer to one object "java". If one reference variable changes the value of the object, it will be affected to all the reference variables. That is why string objects are immutable in java.

Going further.. If you think we forgot about the third way of comparing strings that is with the help of *compareTo()* then certainly we haven't….So let's see how to do it.

The String compareTo() method compares values lexicographically and returns an integer value that describes if first string is less than, equal to or greater than second string.

Suppose s1 and s2 are two string variables. If:

- **s1 == s2** :0
- **s1 > s2** :positive value
- **s1 < s2** :negative value

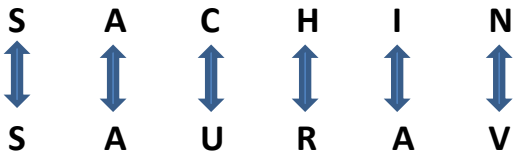
Confused??? Let's understand with example.

Case1:

S1 = “SACHIN”

S2 =“SAURAV”

Comparison happens as shown below



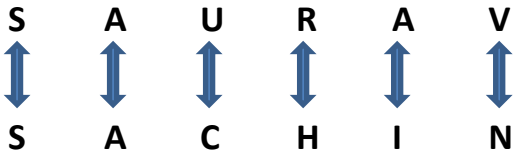
Each character is compared in **lexical order**. In the above example S,A are same in both string but when C and U are compared we know that **C comes before U** therefore **s1 is considered lesser than s2**. Which will give **negative number** when printed on screen.

Case2:

S1= “SAURAV”

S2= “SACHIN”

Comparison happens as shown below



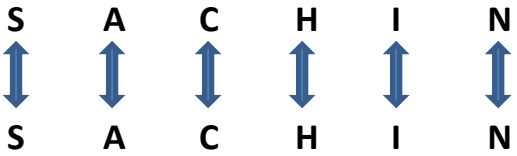
In the above example S,A are same in both string but when U and C are compared we know that **U comes after C** therefore **s1 is considered greater than s2**. Which will give **positive number** when printed on screen.

Case3:

S1 = “SACHIN”

S2 =“SACHIN”

Comparison happens as shown below



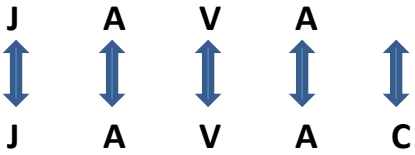
Above we see that all the **characters are same** in both the strings. Therefore when compared character by character there is **no character greater or smaller** than the other which means it will give the result as 0 when printed on the screen.

Case4:

S1= "JAVA"

S2= "JAVAC"

Let's now see what happens when the size of the string is different.



We can see that both the strings contain "JAVA" but **the second string has extra character C**. Therefore **second string is considered to be greater than the first**. And when printed on screen we will get **negative number**.

There are many such string methods, let's have a look at them…

contains() :

The **java string contains()** method searches the sequence of characters in this string. It returns *true* if sequence of char values are found in this string otherwise returns *false*.

Syntax: string_name.contains("sequence");

Returns: true if sequence of char value exists, otherwise **false**.

Throws a **NullPointerException** if the sequence is null.

endsWith() :

The **java string endsWith()** method checks if this string ends with given suffix. It returns true if this string ends with given suffix else returns false.

Syntax: string_name.endsWith("sequence or character");

Returns: true or false.

indexOf() :

The **java string indexOf()** method returns index of given character value or substring. If it is not found, it returns -1. The index counter starts from zero.

There are 4 types of indexOf method in java. The signature of indexOf methods are given below:

No.	Method	Description
1	int indexOf(int ch)	Returns index position for the given char value
2	int indexOf(int ch,int fromIndex)	Returns index position for the given char value and from index
3	int indexOf(String substring)	Returns index position for the given substring
4	int indexOf(String substring, int fromIndex)	Returns index position for the given substring and from index

Parameters:

- **ch**: char value i.e. a single character e.g. 'a'
- **fromIndex**: index position from where index of the char value or substring is retured
- **substring**: substring to be searched in this string

Returns: index of the string

charAt() :

The **java string charAt()** method returns *a char value at the given index number*.

The index number starts from 0 and goes to n-1, where n is length of the string. It returns **StringIndexOutOfBoundsException** if given index number is greater than or equal to this string length or a negative number.

Returns : char value

startsWith() :

The **java string startsWith()** method checks if this string starts with given prefix. It returns true if this string starts with given prefix else returns false.

Syntax :
public boolean startsWith(String prefix)
public boolean startsWith(String prefix, int offset)

parameters : prefix : Sequence of character

Returns: true or false

substring() :

The **java string substring()** method returns a part of the string.

We pass begin index and end index number position in the java substring method where start index is inclusive and end index is exclusive. In other words, start index starts from 0 whereas end index starts from 1.

There are two types of substring methods in java string.

Syntax:

1. **public** String substring(**int** startIndex)
2. **public** String substring(**int** startIndex, **int** endIndex)

parameters:

- **startIndex** : starting index is inclusive
- **endIndex** : ending index is exclusive

Returns: specified string

Throws: StringIndexOutOfBoundsException if start index is negative value or end index is lower than starting index.

toLowerCase() :

The **java string toLowerCase()** method returns the string in lowercase letter. In other words, it converts all characters of the string into lower case letter.

The toLowerCase() method works same as toLowerCase(Locale.getDefault()) method. It internally uses the default locale.

Syntax:

- **public** String toLowerCase()
- **public** String toLowerCase(Locale locale)

The second method variant of toLowerCase(), converts all the characters into lowercase using the rules of given Locale.

Returns: string in lowercase

toUpperCase() :

The **java string toUpperCase()** method returns the string in uppercase letter. In other words, it converts all characters of the string into upper case letter.

The toUpperCase() method works same as toUpperCase(Locale.getDefault()) method. It internally uses the default locale.

Syntax:

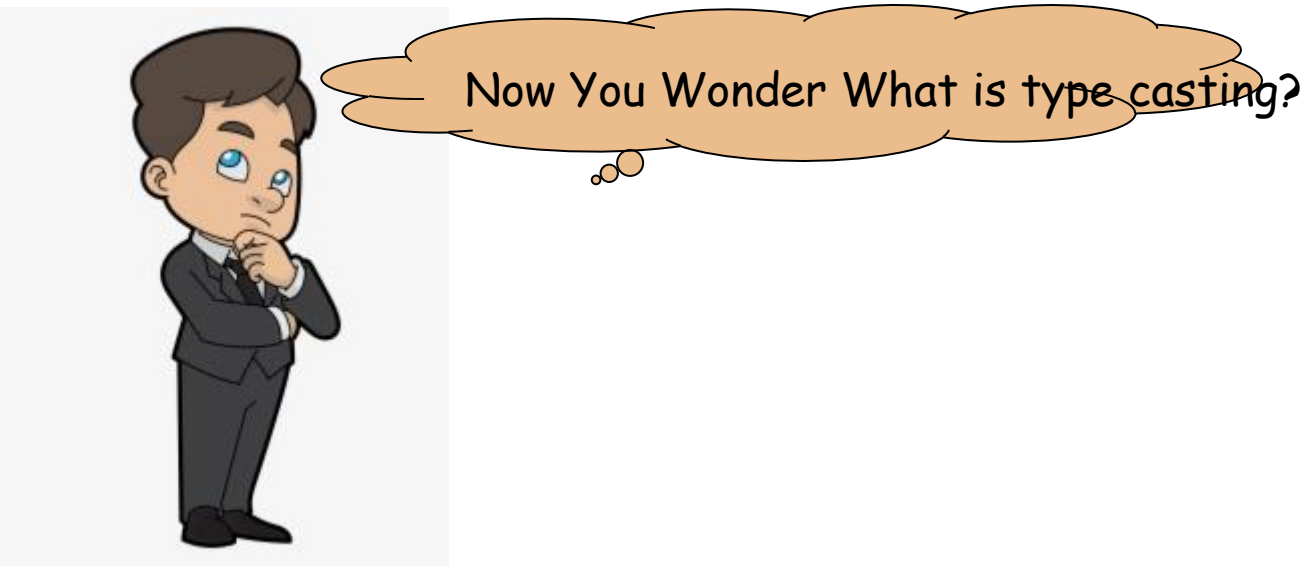
- **public** String toUpperCase()
- **public** String toUpperCase(Locale locale)

The second method variant of toUpperCase(), converts all the characters into uppercase using the rules of given Locale.

Returns: string in uppercase

There are definitely many other methods which you will be exploring once you start practicing.

Type casting in java:



Type casting is a process of converting one type of data to another

In Java, there are two types of casting:

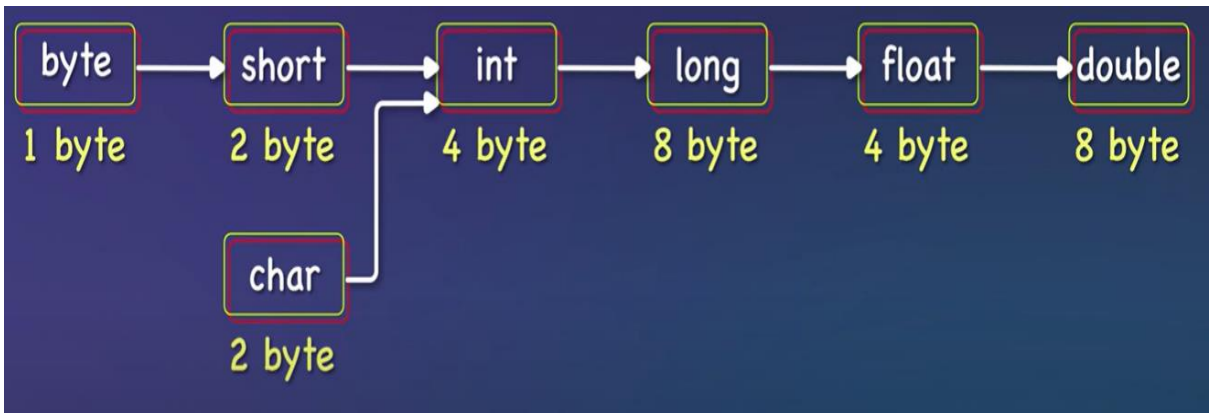
Implicit casting (automatically) - converting a smaller type to a larger type size
byte -> short -> char -> int -> long -> float -> double
Explicit casting (manually) - converting a larger type to a smaller size type

Implicit type Casting:

When a smaller data type is converted to a larger data type, the conversion is automatically performed by **the java compiler** and is referred to as implicit type casting.

Advantage: No loss of precision

Consider the **Implicit type casting** chart given below to understand this:



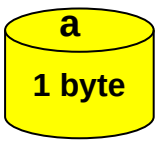
Orders of Implicit Type-Casting for Primitives

Let us consider a code snippet to understand this:

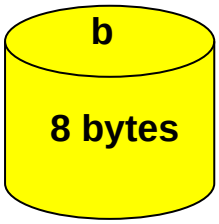
```
byte a = 45;
double b;
b =a;
```

Let us understand implicit type casting using the above code snippet

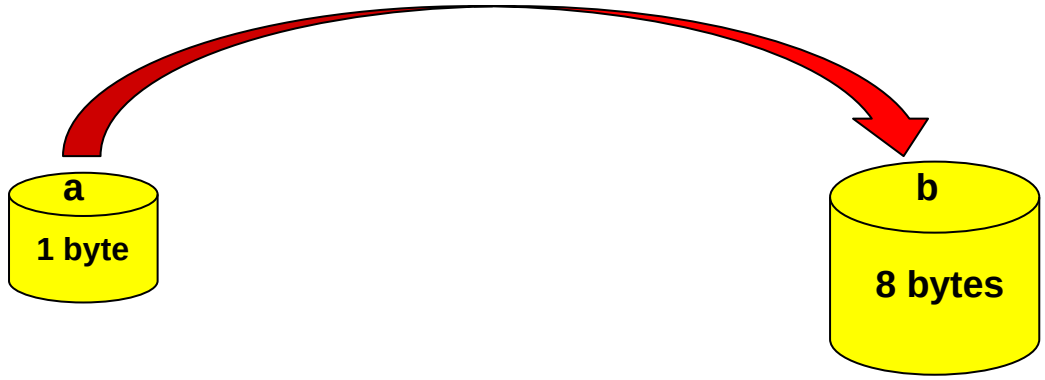
a is a variable of type byte whose size is 1 byte



b is a variable of type byte whose size is 8 byte



b = a; we are now trying to store the data present in a into b;
a is of type byte and can store 1 byte. b is of type double and can store 8 bytes. We are trying to store data of smaller size into larger size.



This conversion is implicitly done without user interaction and hence it is referred to as implicit type casting

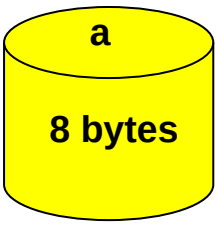
Explicit type Casting:

When a larger data type is converted to a smaller data type, the **conversion is not automatically performed** by **the java compiler** and must be done by **programmer explicitly** and hence it is referred to as explicit type casting

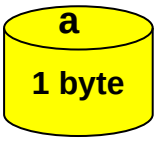
Let us consider a simple code snippet to understand this, the way we understood explicit type casting

```
double a = 45.5;
byte b;
b = a;
```

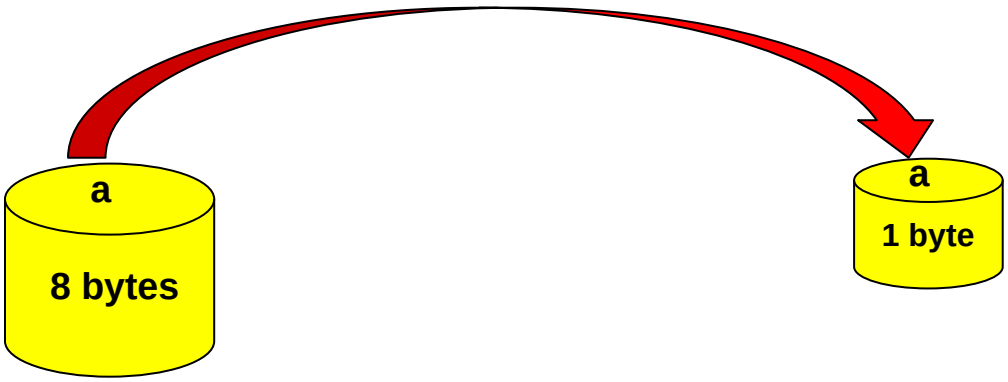
a is a variable of type double whose size is 8 bytes.



b is a variable of type byte whose size is 1 byte.



b=a; will give you **error** as you are trying to store a larger type of data into a smaller type.



The above conversion will result in error as loss of precision occurs. To get the error free output, we have to explicitly convert the data as shown below

```
double a = 45.5;
byte b;
b = (byte)a;
```

b is of type byte and it will only store 45 and 0.5 is lost during the conversion which is the disadvantage of explicit type casting.

What is a variable ?

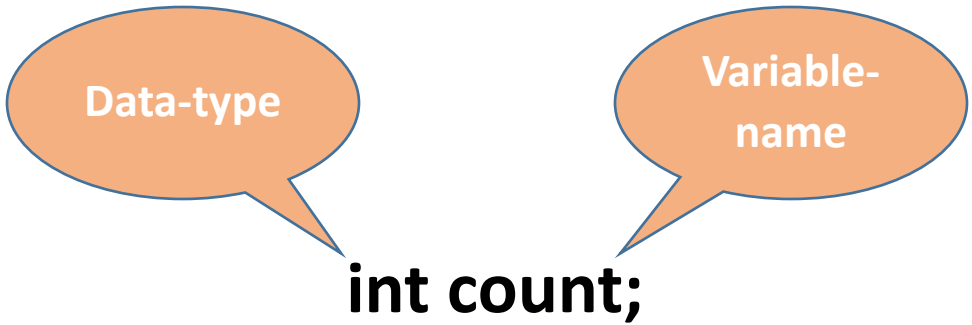
- A variable which holds value, during the life of a Java program.

- Every variable is assigned a **data type** which designates the type and quantity of value it can hold.

- In order to use a variable in a program you to need to perform 2 steps
--Variable Declaration
--Variable Initialization

Variable Declaration

To declare a variable, you must specify the data type & give the variable a unique name.



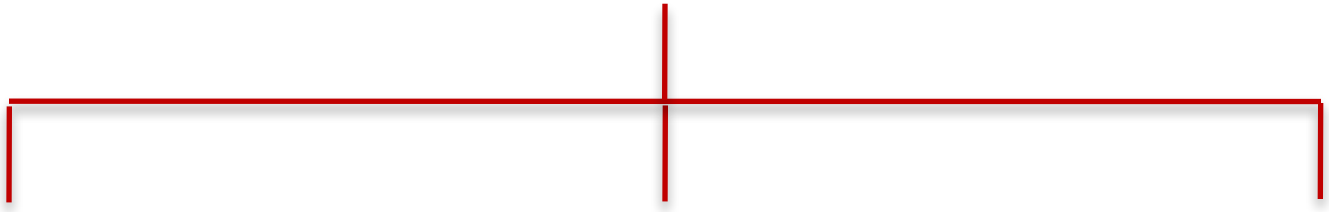
You can combine variable declaration and initialization.

int count=100;

NAMING CONVENTION OF VARIABLES

- can start with underscore(‘_’) but not with digits.
- Should be mnemonic i.e, designed to indicate to the casual observer the intent of its use.
- Can use _(underscore), digits and letters.
- Should not use any reserved word.

TYPES OF VARIABLES



Local variables

Instance variables

Static variables

Consider this code snippet

```
class Face99
{
        int data = 99; //instance variable
        static int a = 1; //static variable
        void method()
        {
                int b = 90; //local variable
        }
}
```

VARIABLES

A variable is a container which holds the value while the Java program is executed. A variable is assigned with a data type.

Variable is a name of memory location.

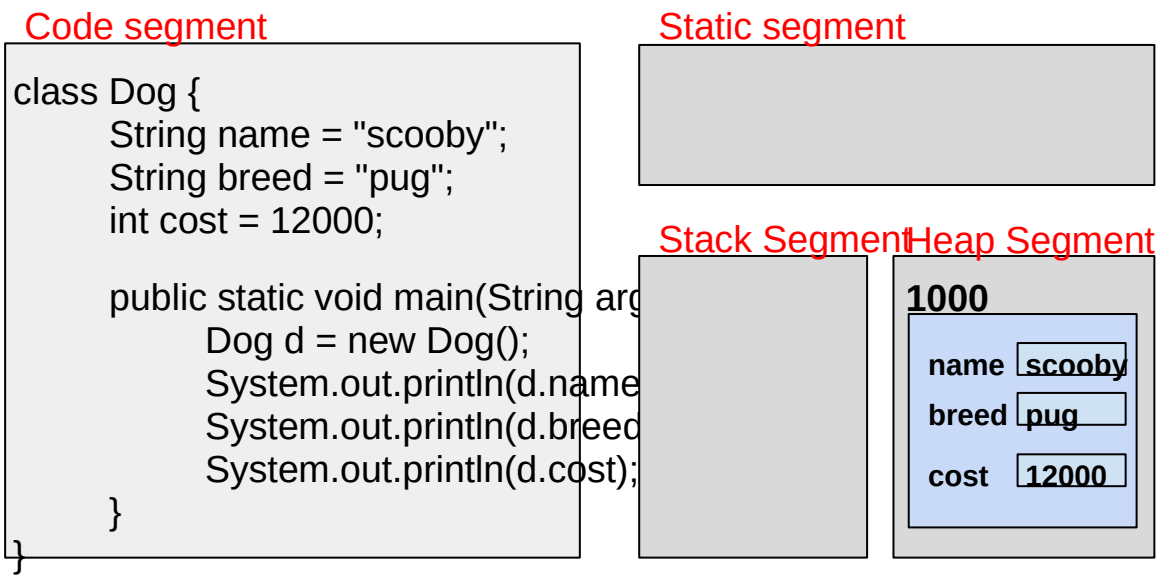
There are three types of variables in java:

- local
- Instance
- Static

Now let's understand about Local and Instance Variables

Instance Variables:

The variables which will get created inside the class outside the methods are called instance variables

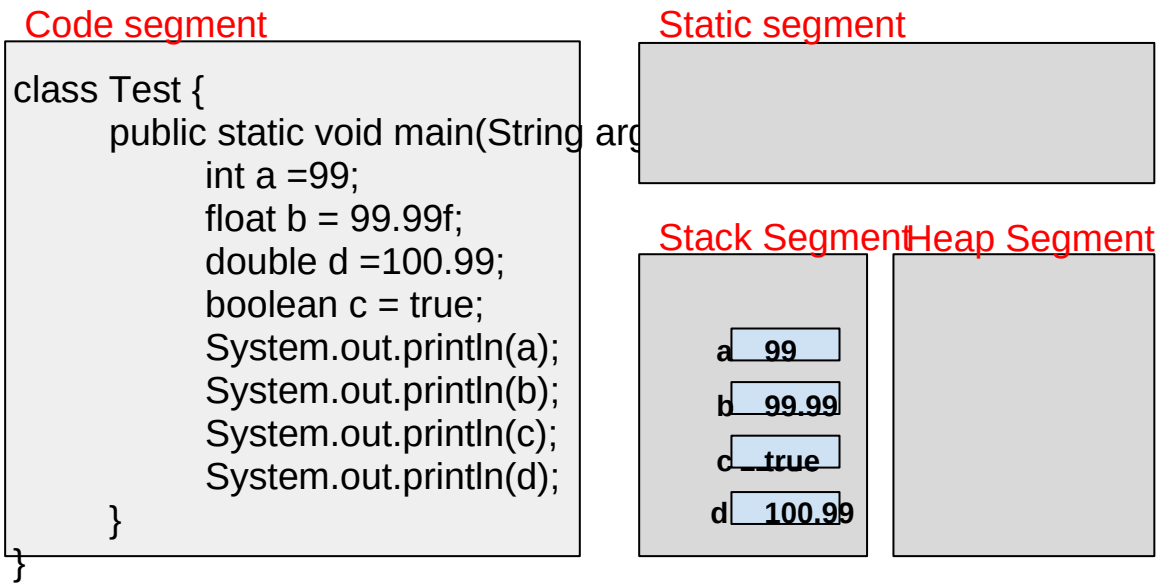


Here control of execution will start from the main method. In the main method when a new keyword is called an object is created in the heap segment and memory for instance variables name, breed and cost is allocated the default values for the

variables is allocated then the values that is assigned will get allocated i.e name=”scooby”, breed = “pug”, cost = 12000.

Local Variables:

The variables which will get created inside the class inside the methods are called local variables



The control of execution will start from the main method and the memory for variables a,b,c,d is allocated in the stack segment and values will be assigned. Default values for local variables will not be assigned by jvm.

Introduction to Wrapper Class:

Let us first understand why wrapper class?

Java is an impure object-oriented programming language because it supports **primitive data types.**
You may be wondering; does it really matter if java is pure or impure object oriented

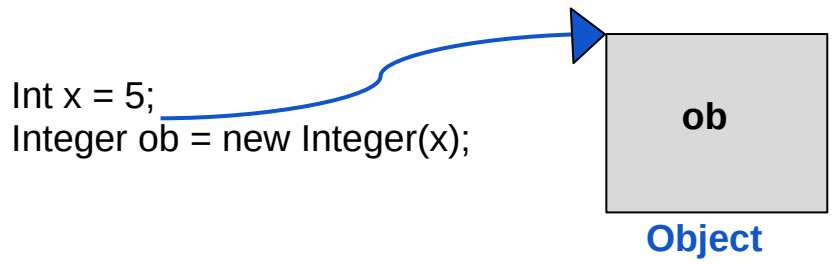
To understand this, you must know an important fact about primitive data types which is **Primitive data types are not treated as object in java**

However, 100% pure object-oriented programs can be written in java using **wrapper class**
Now you understood why wrapper class lets us understand what is wrapper class.

What is Wrapper class?

The **Wrapper class in java is a class** that provides the mechanism **to convert primitive data types into objects and objects into primitive data type.**

Well how if you wonder, consider the below example.



In the above example **int** is a **primitive data** type and **Integer** is a **wrapper class**. Let us have a look at one more example of character data type.

If you are using a single character value, you will use the primitive char type

```
char ch = 'a'
```

There are times, however, when you need to use a char as an object - for example, as a method argument where an object is expected.

The java programming language provides a wrapper class that "wraps" the char in a character object for this purpose. An object of type character contains a single field, whose type is char. This character class also offers a number of useful class (i.e. static) methods for manipulating characters.

You can create a character object with character constructor.

```
Character characterObj = new Character('a');
```

Advantage of wrapper class: The program becomes pure object oriented

Disadvantage of wrapper class: Execution speed decreases.

Advantage of primitive data type: Faster in execution

Disadvantage of primitive data type: The program becomes impure object oriented.

Operators

Unary operators

Operator	Description	Example
++	Increases the value of a variable by 1	++x(prefix) x++(postfix)
--	Decreases the value of a variable by 1	--x(prefix) x--(postfix)
+	Used for giving positive values	+ x
-	Used for negating the values	- x
~	Negating an expression	~ x
!	Inverting the value of a boolean	! x

Arithmetic operators

Operator	Description	Example
+	Adds two values	x + y
-	Subtracts one value from another	x - y
*	Multiplies two values	x * y
/	Returns the division quotient	x / y
%	Returns the division remainder	x % y

```
1  class OperatorExample{
2      public static void main(String args[]){
3        int a=10;
4        int b=5;
5        System.out.println(a+b);
6        System.out.println(a-b);
7        System.out.println(a*b);
10       System.out.println(a/b);
11       System.out.println(a%b);
12     }
13 }
```

output

```
15
5
50
2
0
```

Relational operators

Operator	Description	Example
==	Returns true if left hand side is equal to right hand side	x == y
!=	Returns true if left hand side is not equal to right hand side	x != y
<	Returns true if left hand side is less than right hand side	x < y
<=	Returns true if left hand side is less than or equal to right hand side	x < =y
>	Returns true if left hand side is greater than right hand side	x >=y
>=	Returns true if left hand side is greater than or equal to right hand side	x >= y

```
1  public class Test {
2     public static void main(String args[]){
3        int a = 10;
4        int b = 20;
5        System.out.println(a>b);
6        System.out.println(a<b);
7        System.out.println(a>=b);
10       System.out.println(a<=b);
11       System.out.println(a==b);
12       System.out.println(a!=b);
13    }
14 }
```

output

```
false
true
false
true
false
true
```

Bitwise operators

Operator	Description	Example
&	Returns bit by bit AND of input values	x & y
|	Returns bit by bit OR of input values	x | y
^	Returns bit by bit XOR of input values	x ^ y
~	Returns the one's compliment representation of the input value	~ x
<<	shifts the bits of the number to the left and fills 0 on voids left as a result	x << 2
>>	shifts the bits of the number to the right	x >> 2
>>>	shifts the bits of the number to the right and fills 0 on voids left as a result	x >>>2

```java
public class Main {
    public static void main(String args[]){
        int a = 10;
        int b = 20;
        System.out.println(a&b);
        System.out.println(a|b);
        System.out.println(~a);
        System.out.println(a<<2);
        System.out.println(a>>2);
        System.out.println(a>>>2);
    }
}
```

output

```
0
30
-11
40
2
2
```

Logical operators

Operator	Description	Example
&&	Returns true if both statements are true	x < 5 && x < 10
||	Returns true if one of the statements is true	x < 5 || x < 4
!	Reverse the result, returns false if the result is true	!(x < 5 && x < 10)

```
1  public class Test {
2     public static void main(String args[]){
3        boolean a = true;
4        boolean b = false;
5        System.out.println(a&&b);
6        System.out.println(a||b);
7        System.out.println(!(a && b));
10    }
11 }
```

output

```
false
true
true
```

```
1  class OperatorExample{
2     public static void main(String args[]){
3        int a=2;
4        int b=5;
5        int min=(a<b)?a:b;
6        System.out.println(min);
7     }
10 }
```

output

```
2
```

Assignment operators

Operator	Description	Example
=	Assigns values from right side operands to left side operand.	C = A + B
+=	Adds right operand to the left operand and assign the result to left operand.	C += A
-=	Subtracts right operand from the left operand and assign the result to left operand.	C -= A
*=	Multiplies right operand with the left operand and assign the result to left operand.	C *= A
/=	Divides left operand with the right operand and assign the result to left operand.	C /= A
%=	Takes modulus using two operands and assign the result to left operand.	C %= A

Assignment operators

Operator	Description	Example
<<=	Left shift AND assignment operator	C <<= 2
>>=	Right shift AND assignment operator	C >>= 2
&=	Bitwise AND assignment operator	C &= 2
^=	Bitwise exclusive OR and assignment operator	C ^= 2
|=	Bitwise inclusive OR and assignment operator	C |= 2

```
1  class OperatorExample{
2    public static void main(String[] args){
3        int a=10;
4        a+=3;
5        System.out.println(a);
6        a-=4;
7        System.out.println(a);
10       a*=2;
11       System.out.println(a);
12       a/=2;
13       System.out.println(a);
14   }
15 }
```

output

```
13
9
18
9
```

Operator	Description	Associativity

`()`
`[]`
`.`
	method invocation
	array subscript
	member access/selection
		left-to-right

`++` `--`	unary postfix increment unary postfix decrement	right-to-left

`++`
`--`
`+`
`-`
`!`
`~`
`( type )`
`new`
	unary prefix increment
	unary prefix decrement
	unary plus
	unary minus
	unary logical negation
	unary bitwise complement
	unary cast
	object creation
		right-to-left

`*`
`/`
`%`
	multiplication
	division
	modulus (remainder)
		left-to-right

`+` `-`	addition or string concatenation subtraction	left-to-right

`<<`
`>>`
`>>>`
	left shift
	arithmetic/signed right shift (sign bit duplicated)
	logical/unsigned right shift (zero shifted in)
		left-to-right

`<`
`<=`
`>`
	less than
	less than or equal to
	greater than
		left-to-right

`>`
`>=`
`instanceof`
	greater than
	greater than or equal to
	type comparison	

`==` `!=`	is equal to (equality) is not equal to (inequality)	left-to-right
`&`	bitwise AND boolean logical AND (no short-circuiting)	left-to-right
`^`	bitwise exclusive OR boolean logical exclusive OR	left-to-right
`|`	bitwise inclusive OR boolean logical inclusive OR (no short-circuiting)	left-to-right
`&&`	logical/conditional AND (short-circuiting)	left-to-right
`||`	logical/conditional OR (short-circuiting)	left-to-right
`?:`	conditional/ternary (if-then-else)	right-to-left

`=`
`+=`
`-=`
`*=`
`/=`
`%=`
`&=`
`^=`
`|=`
`<<=`
`>>=`
`>>>=`
	assignment
	addition assignment
	subtraction assignment
	multiplication assignment
	division assignment
	modulus/remainder assignment
	bitwise AND assignment
	bitwise exclusive OR assignment
	bitwise inclusive OR assignment
	bitwise left shift assignment
	bitwise arithmetic/signed right shift assignment
	bitwise logical/unsigned right shift assignment
		right-to-left

Operators

Increment

Increment and Decrement in Java programming let you easily **add** 1 or **subtract** 1 from variable.

To achieve this, we have two different types of operators.

INCREMENT

In Java, the increment unary operator increases the value of the variable by one.

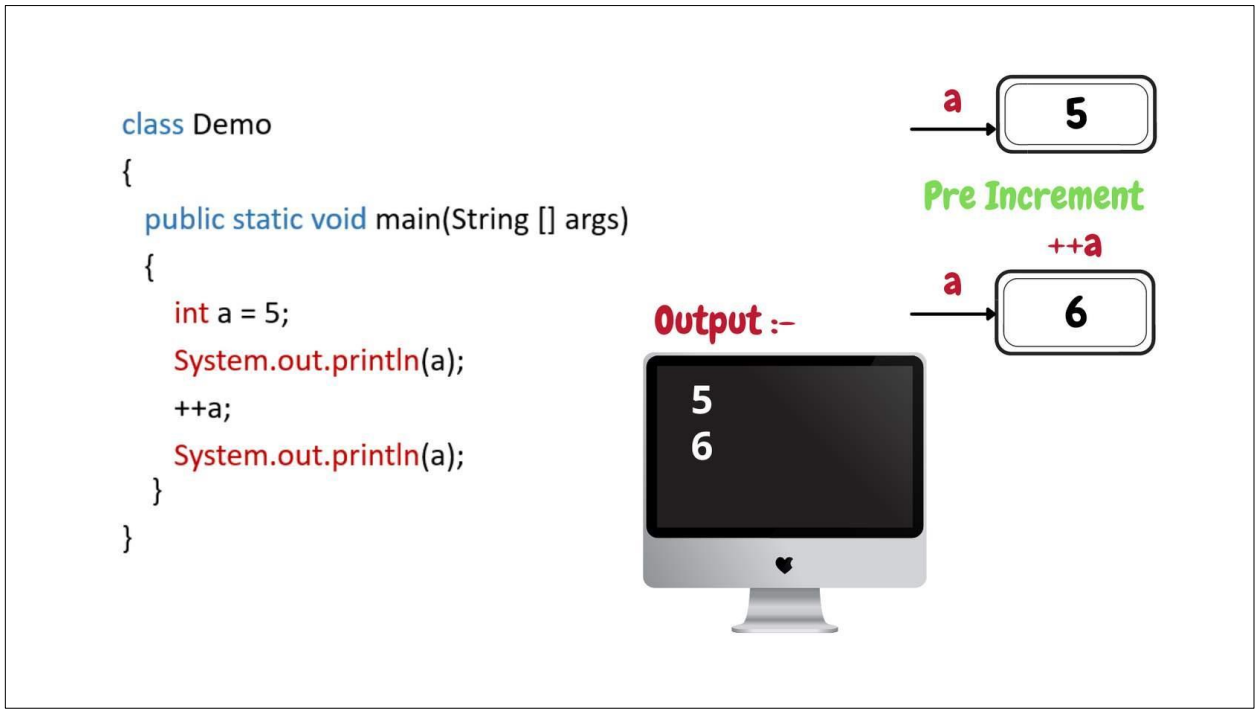
" ++ " is the operator used to increment.

There are two types of Increment

- Pre-Increment.
- Post-Increment.

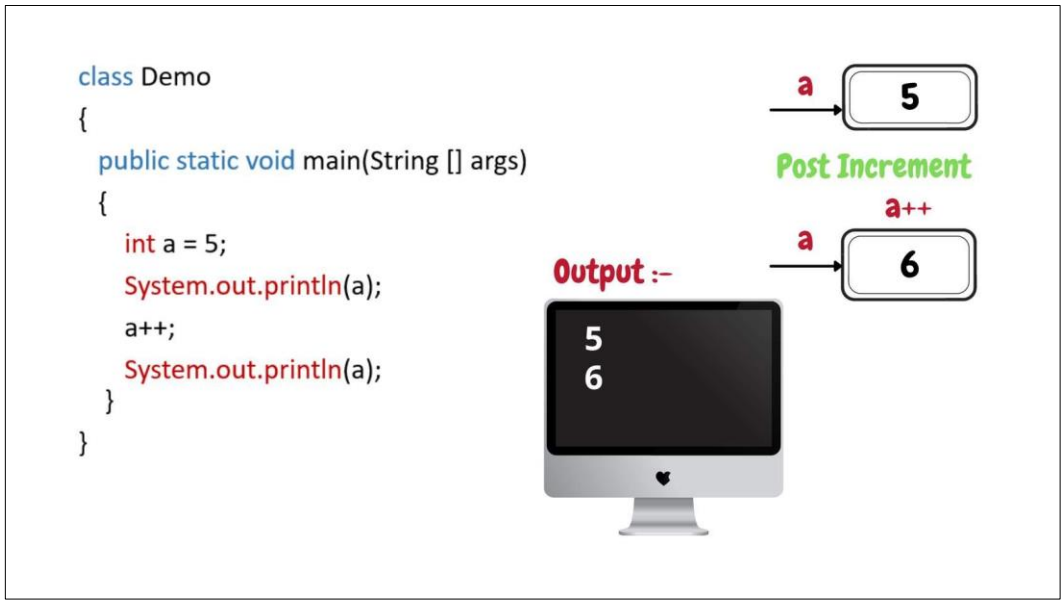
Pre-Increment-

- "++" is written before Variable name.
- Value will be Incremented First and then incremented value is used in expression.



Post-Increment-

- "++" is written after Variable name.
- Value is used in expression first and then gets incremented.



Decrement

In Java, the decrement unary operator decreases the value of the variable by one.

" -- " is the operator used to decrement.

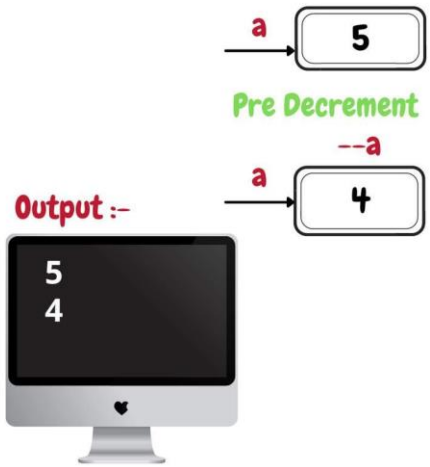
There are two types of Increment

- Post-Decrement.
- Pre-Decrement.

Pre-Decrement-

- "--" is written before Variable name.
- Value is decremented First and then decremented value is used in expression

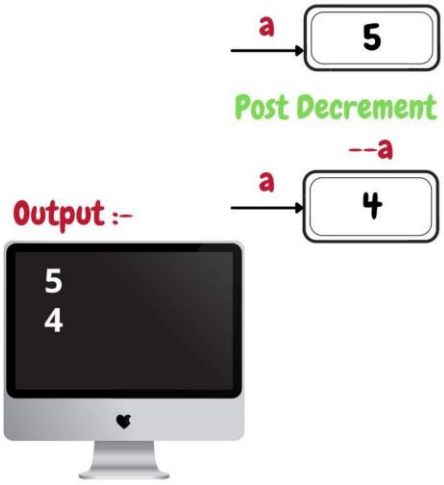
```
class Demo
{
  public static void main(String [] args)
  {
     int a = 5;
     System.out.println(a);
     --a;
     System.out.println(a);
   }
}
```



Post-Decrement-

- "--" is written after Variable name.
- Value is used in expression first and then gets decremented.

```
class Demo
{
  public static void main(String [] args)
  {
     int a = 5;
     System.out.println(a);
     a--;
     System.out.println(a);
   }
}
```



Problems on Increment and Decrement.

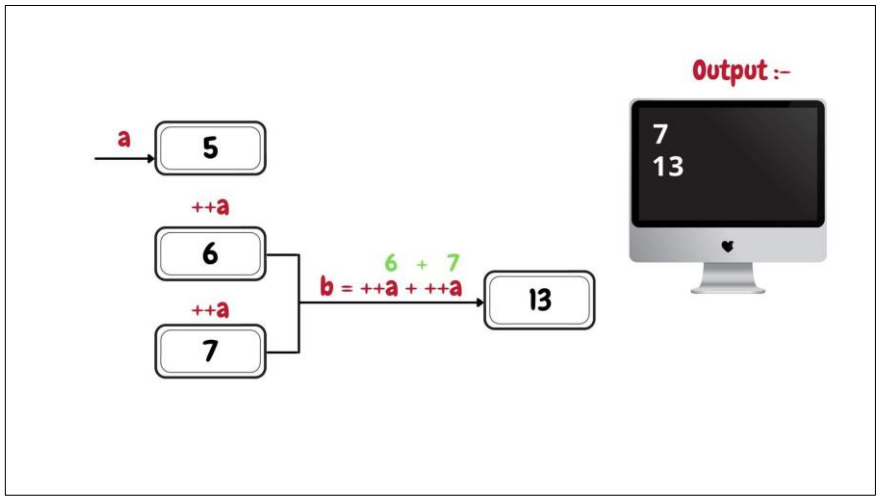
Question 01

Consider a = 5, print a and b

b = ++a + ++a;

Code:

```
class Demo
{
   public static void main (String [] args)
   {            int a = 5 ;
                int b;
                b = ++a + ++a;
                System.out.println(a);
                System.out.println(b);
    }
}
```



Question 02

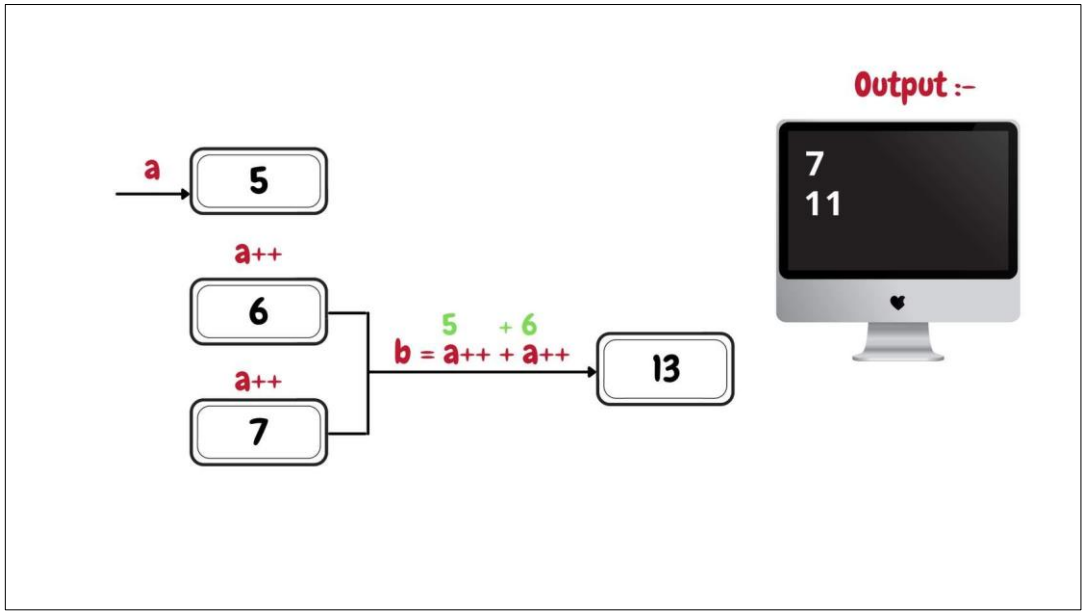
Consider a = 5, print a and b

b = a++ + a++;

Code:

```
class Demo
{
   public static void main (String [] args)
```

```
   {            int a = 5 ;
                int b;
                b = a++ + a++;
                System.out.println(a);
                System.out.println(b);
    }
}
```



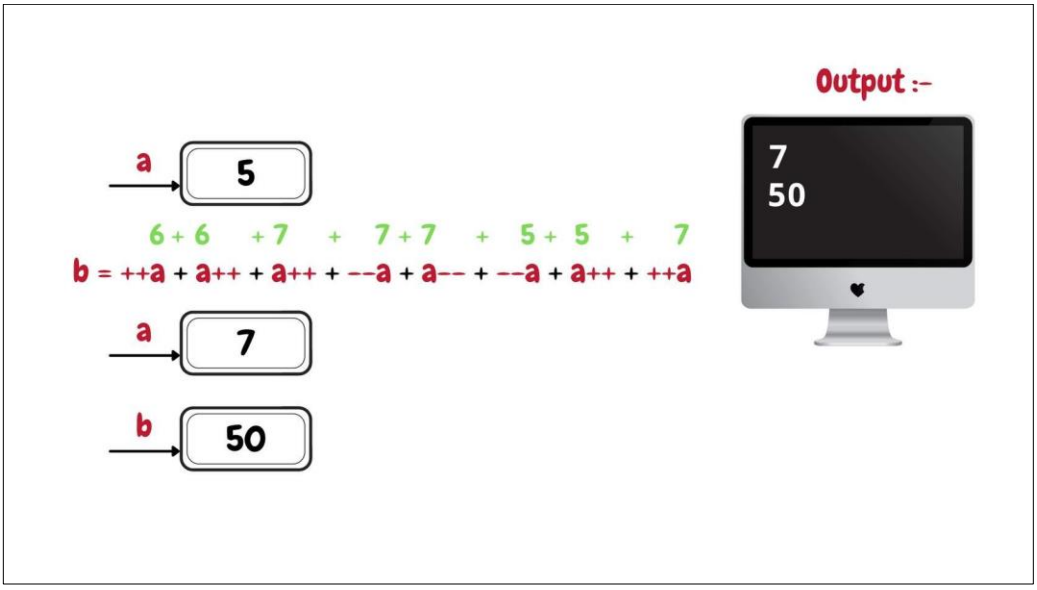
Question 03

Consider a = 5, print a and b

b = ++a + a++ + a++ + --a + a-- + --a + a++ + ++a;

Code:

```
class Demo
{
   public static void main (String [] args)
   {            int a = 5 ;
                int b;
                b = ++a + a++ + a++ + --a + a-- + --a + a++ + ++a;
                System.out.println(a);
                System.out.println(b);
    }
}
```


Output :-
a
5
6 + 6 + 7 + 7 + 7 + 5 + 5 + 7
b = ++a + a++ + a++ + --a + a-- + --a + a++ + ++a
a
7
b
50
7
50

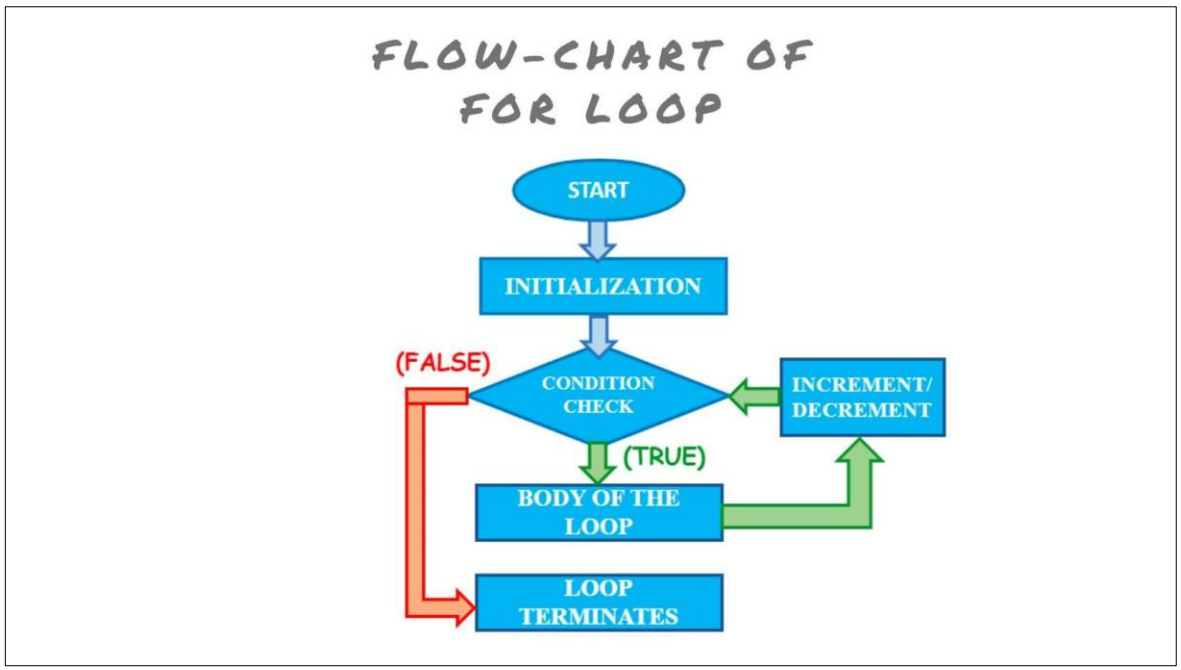
Loops


```
class Demo
{
  public static void main(String [] args)
  {
    System.out.println("JAVA");
    System.out.println("JAVA");
    System.out.println("JAVA");
    System.out.println("JAVA");
    System.out.println("JAVA");
  }
}
```

Imagine you have to print "JAVA" 100 times?

Are you going to type the statement 100 times?

Here comes the loop to rescue the programmer and make the job easy. Most fundamental and basic loop is known as FOR loop.

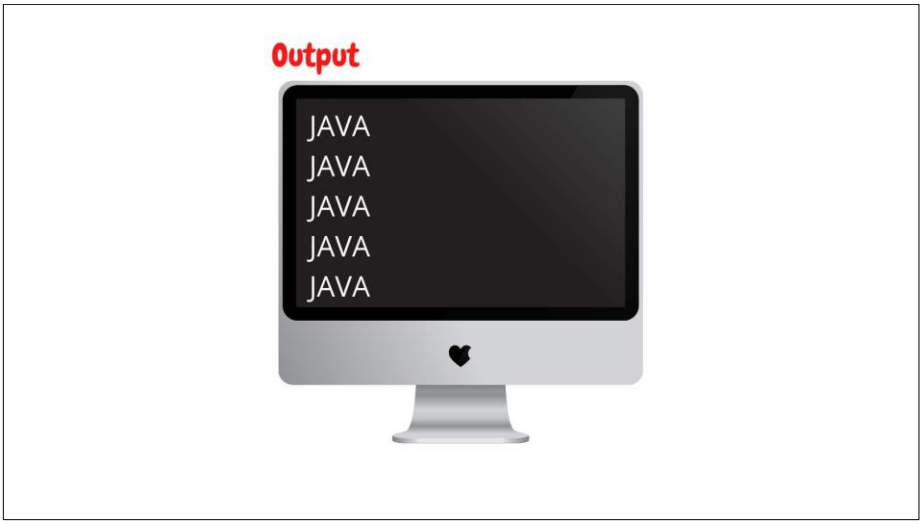


Syntax:

for (initialization; condition; increment/ decrement)

Code:

```
class Demo
{
   public static void main (String [] args)
   {
         int i;
         for ( i=1; i<=5; i++)
         {
            System.out.println("JAVA");
         }
    }
}
```



Value type assignment

```
class Test2
{
	public static void main(String[] args)
	{
		int x = 100; // Assigning the value to the variable x
		int y; // Declaring the variable y

		y = x;  // assigning the value present in x to y

		System.out.println(x); // print the value of x
		System.out.println(y); // print the value of y

		x = 200;  // modifying the value of x

		System.out.println(x); // print the value of x
		System.out.println(y); // print the value of y
	}
}
```

Output:

```
100
100
200
100
```

If you observe from the above output, if you change the value present in one variable it will not affect other variable

Reference type assignmemt

- **Let's create two instance variable name and cost and assign values to those variables**

```
class Car
{
	String name; // instance variable name is created of type string
	int cost; // instance variable cost is created of type int
}

class Test2
{
	public static void main(String[] args)
	{
		Car x = new Car(); // car class object is created
		x.name = "maruthi"; // assigning value to the instance variable name using object reference
		x.cost = 200000; // assigning value to the instance variable cost using object reference

		System.out.println(x.name); // print the value present in instance variable
		System.out.println(x.cost); // print the value present in instance variable

	}
}
```

Output:

```
maruthi
200000
```

- Let's create one more object reference y of type car and assign the reference present in x to y.

```
class Car
{
	String name; // instance variable name is created of type string
	int cost;	// instance variable cost is created of type int
}

class Test2
{
	public static void main(String[] args)
	{
		Car x = new Car(); // car class object is created
		x.name = "maruthi"; // assigning value to the instance variable name using object reference
		x.cost = 200000; // assigning value to the instance variable cost using object reference

		System.out.println(x.name); // print the value present in instance variable name
		System.out.println(x.cost); // print the value present in instance variable cost

		Car y;  // create an object reference of type y
		y = x;  // assign the reference present inside x to y

		System.out.println(y.name); // print the value present in instance variable name using object reference y
		System.out.println(y.cost); // print the value present in instance
```

```
variable cost using object reference y

        }
}
```

Output:

```
maruthi
200000
maruthi
200000
```

If you observe from the above output when you print the values using object reference x and y it is giving you same result it is because both are pointing to same object

- **Now both the object reference x and y are pointing to the same object. Let's try to modify the value of instance variable using one reference**

```
class Car
{
        String name;
        int cost;
}

class Test2
{
        public static void main(String[] args)
        {
                Car x = new Car();
                x.name = "maruthi";
                x.cost = 200000;

                System.out.println(x.name);
```

```
                System.out.println(x.cost);

                Car y;
                y = x;

                System.out.println(y.name);
                System.out.println(y.cost);

                y.name = "BMW"; // now let's modify the instance variable name using object reference y
                y.cost = 500000; // now let's modify the instance variable cost using object reference y

                System.out.println(x.name); // print the value of name using object reference x
                System.out.println(x.cost); // print the value of cost using object reference x
        }
}
```

Output:

```
maruthi
200000
maruthi
200000
BMW
500000
```

If you observe from the above output if you modified the value using one object reference it will effect all other references pointing to the same object

Pass by value

- **Create a method which accepts two parameters and return the result**

```
class Calculator
{
     int c; // create an instance variable c of type int

          // define the method add which accepts two
parameters perform addition operation and return the
result
     int add(int a, int b)
     {
          c = a + b;
          return c;
     }

     public static void main(String[] args)
     {
          Calculator calc = new Calculator(); // create
an object of class calculator
          int num1 = 50; // assign value 50 to the
variable num1
          int num2 = 40; //assign the value 40 to the
variable num2
          int res = calc.add(num1, num2); // call add
method and store the returned value store it in res
          System.out.println(res); //print res

          }
}
```

Output:

```
90
```

```
class Calculator
{
     int c;
     int add(int a, int b)
     {
          a = 500; // here local variable value is
assigned with value 500
          b = 400; // here local variable value is
assigned with value 400
          c = a + b; //Addition of two variable a and b
is done and store the result in c
          return c; // result the value present in c
     }

     public static void main(String[] args)
     {
          Calculator calc = new Calculator(); // create
an object of class
          int num1 = 50; // declare local variable num1
and assign the value 10
          int num2 = 40; // declare local variable num1
and assign the value 10
          int res = calc.add(num1, num2); //call the
method by passing the value present in num1 and num2
          System.out.println(res); //print res that is
returned by the method

          }
}
```

Output:

900

- **Now let's try to create the method which accept the reference of type car and modify the value of instance variable inside that method**

```
class Car
{

	String name;
	int cost;

	void modifyCar(Car y)
	{
		y.name = "BMW";
		y.cost = 500000;
	}
}

class Test2
{
	public static void main(String[] args)
	{
		Car x = new Car();
		x.name = "maruthi";
		x.cost = 200000;

		System.out.println(x.name);
		System.out.println(x.cost);

		x.modifyCar(x);

		System.out.println(x.name);
		System.out.println(x.cost);

	}
}
```

Output:

```
maruthi
200000
BMW
500000
```

Here if you observe from the above output the you have updated the value by using the reference y now if you try to access it using the reference x you are getting updated value it is because of pass by reference